

The data consumer creates a table in DynamoDB to maintain state information (such as checkpoints and worker-shard mapping) for the application. Each application has its own DynamoDB table. The data records, once processed, can then be sent to a more concrete storage like S3 or DynamoDB. Or, they can be used for real-time analytics and discarded later.

Kinesis Client Library (KCL)

Applications can also use the Kinesis Client Library (KCL) to simplify parallel processing of the stream. The library takes care of the complex tasks such as load-balancing across multiple instances, responding to failures, checkpointing, and reacting to resharding. The KCL enables you to focus on writing record processing logic.

The data consumer provides the KCL with position of the stream that it wants to read from, the library then calls `GetShardIterator` on behalf of the consumer. The consumer component also provides the client library with the processing logic for the incoming records using an important KCL interface called `IRecordProcessor`. The KCL calls `GetRecords` on behalf of the consumer and then processes those records as specified by `IRecordProcessor`.

Elastic Map Reduce (EMR)

MapReduce is a framework for processing large datasets in parallel using a large number of instances (nodes), collectively referred to as a cluster or a grid (based on location and specification details). MapReduce jobs can run on both file system records as well as database entries. A typical MapReduce job consists of two steps, Map and Reduce. In the “Map” step, each worker (node) applies the “`map()`” function to the data, and writes the output to a temporary storage. MapReduce supports distributed processing of the map and reduction operations. Provided that each mapping operation is independent of the others, all maps can be performed in parallel – though in reality this is limited by the number of CPUs allocated to each source.



Once you have your data ingested, use MapReduce to sort and filter it.